

High-Level Browser Architecture

A browser is software that helps users access websites.

Examples:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

A browser mainly consists of:

A. User Interface (UI)

The visible part of the browser.

Examples:

- Address bar
- Back button
- Forward button
- Refresh button
- Bookmarks

When you type:

`https://www.google.com`

you are interacting with the UI.

B. Browser Engine

Acts as a bridge between the UI and rendering engine.

It receives instructions from the user and tells other browser components what to do.

Example:

You click Refresh.

Browser Engine says:

"Reload this page."

C. Rendering Engine

Responsible for displaying web pages.

It reads:

- HTML
- CSS

and converts them into the visual webpage you see.

Example:

HTML:

```
<h1>Hello</h1>
```

CSS:

```
h1 {  
  color: red;  
}
```

Rendering Engine shows:

Hello (in red color)

D. JavaScript Engine

Executes JavaScript code.

Examples:

- Chrome → V8 Engine
- Firefox → SpiderMonkey

Example:

```
alert("Welcome");
```

JavaScript Engine executes it and displays:

```
Welcome
```

popup.

E. Networking Layer

Handles communication with servers.

Responsible for:

- HTTP requests
- HTTPS requests
- DNS lookup
- TCP connection

Example:

When opening Google, the Networking Layer sends a request to Google's server.

F. Storage

Stores website data.

Examples:

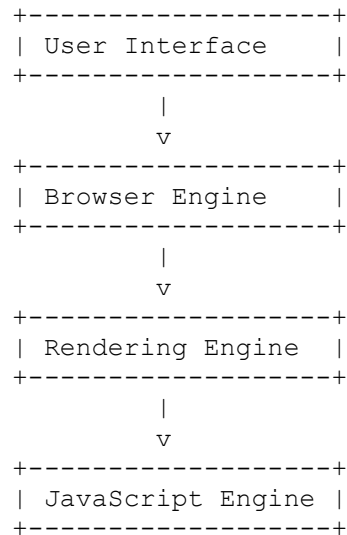
- Cookies
- Local Storage
- Session Storage
- IndexedDB

Example:

```
localStorage.setItem("name", "Gaurav")
```

Browser saves data locally.

Browser Architecture Diagram



Networking Layer
Storage

2. DNS Resolution

DNS = Domain Name System

DNS converts a domain name into an IP address.

Humans remember:

`www.google.com`

Computers understand:

`142.250.195.78`

DNS acts like a phonebook.

Example

Suppose you enter:

`www.google.com`

Browser doesn't know the IP.

It asks DNS:

What is the IP of google.com?

DNS replies:

142.250.195.78

Now browser knows where to connect.

DNS Resolution Flow

```
Browser
  |
  v
DNS Resolver
  |
  v
Root Server
  |
  v
.com Server
  |
  v
Google DNS Server
  |
  v
IP Address Returned
```

Real-Life Example

Imagine:

```
Contact Name: Rahul
Phone Number: 9876543210
```

You remember Rahul's name, not the number.

DNS works exactly like this.

```
Google.com = Name
IP Address = Phone Number
```

3. TCP/IP Handshake

After DNS gives the IP address, the browser must establish a connection.

TCP = Transmission Control Protocol

TCP ensures reliable communication.

Before sending data, both sides perform a 3-Way Handshake.

Three-Way Handshake

Step 1: SYN

Browser says:

```
Can we connect?  
Client --> SYN --> Server
```

Step 2: SYN-ACK

Server replies:

```
Yes, let's connect.  
Server --> SYN ACK --> Client
```

Step 3: ACK

Browser confirms:

```
Great, connection established.  
Client --> ACK --> Server
```

Diagram

```
Client                Server  
  
SYN  ----->  
  
      <----- SYN-ACK  
  
ACK  ----->  
  
Connection Established
```

Real-Life Example

Imagine calling a friend:

You: Hello?
Friend: Yes, I can hear you.
You: Great.

Conversation starts.

That is exactly how TCP handshake works.

4. HTTP and HTTPS

HTTP = HyperText Transfer Protocol

Used for communication between browser and server.

HTTP Example

Request:

```
GET /home HTTP/1.1  
Host: example.com
```

Server Response:

```
200 OK
```

with webpage data.

Problem with HTTP

Data travels in plain text.

Hackers can read it.

Example:

Username: gaurav
Password: 123456

can be intercepted.

HTTPS

HTTPS = HTTP + SSL/TLS Encryption

Data becomes encrypted.

Example:

Instead of:

password=123456

attacker sees:

h7f83hd8s7d9s7

which is unreadable.

HTTPS Flow

```
Browser
|
TLS Handshake
|
Encrypted Connection
|
Server
```

Common HTTP Methods

GET

Fetch data.

GET /users

Example:

Get all users.

POST

Create data.

`POST /users`

Example:

Create a new user.

PUT

Update complete data.

`PUT /users/1`

Example:

Update user information.

DELETE

Delete data.

`DELETE /users/1`

Example:

Delete user.

HTTP Headers

Headers provide extra information.

Example Request:

```
GET /users HTTP/1.1
Host: api.example.com
Authorization: Bearer Token123
Content-Type: application/json
```

Important Request Headers

Host

```
Host: google.com
```

Tells server which website is requested.

Authorization

```
Authorization: Bearer xyz123
```

Used for authentication.

Content-Type

```
Content-Type: application/json
```

Tells server data format.

Important Response Headers

Status Code

```
200 OK
```

Success.

Content-Type

Content-Type: application/json

Response contains JSON.

Set-Cookie

Set-Cookie: token=abc

Browser stores cookie.

Common HTTP Status Codes

200

Success

201

Created

301

Redirect

400

Bad Request

401

Unauthorized

403

Forbidden

404

Not Found

500

Internal Server Error

5. Complete Request-Response Life Cycle

Suppose you open:

`https://www.google.com`

Step 1

User enters URL.

`https://www.google.com`

Step 2

Browser checks cache.

If page exists:

Return cached page

Otherwise continue.

Step 3

DNS Resolution

`google.com`

becomes

`142.x.x.x`

Step 4

TCP Handshake

SYN
SYN-ACK
ACK

Connection established.

Step 5

TLS Handshake (HTTPS)

Secure encrypted connection created.

Step 6

Browser sends HTTP Request

GET /
Host: google.com

Step 7

Server Processes Request

Server:

- Reads request
- Runs backend code
- Queries database if needed

Example:

Node.js
Express
MongoDB

Step 8

Server Sends Response

200 OK

along with:

```
<html>
...
</html>
```

Step 9

Browser Receives Response

Downloads:

- HTML
 - CSS
 - JavaScript
 - Images
-

Step 10

Rendering Engine Creates Page

Browser:

1. Parses HTML
 2. Parses CSS
 3. Executes JS
 4. Builds page
-

Final Result

Google Home Page

appears on screen.

One-Line Interview Answer

When a user enters a URL, the browser checks cache, performs DNS resolution to get the server IP, establishes a TCP connection through a 3-way handshake, creates a secure TLS connection for HTTPS, sends an HTTP request, receives the server response, downloads resources, renders HTML/CSS, executes JavaScript, and finally displays the webpage to the user.

This full flow is one of the most frequently asked browser and web internals questions in MERN, frontend, and full-stack interviews.



Semantic HTML Tags vs Non-Semantic Tags

When building a web page, HTML tags can be divided into **Semantic Tags** and **Non-Semantic Tags**.

Think of HTML as writing a book:

- Semantic tags tell us **what a section means**.
- Non-semantic tags only tell us **how things are grouped**.

1. What are Semantic HTML Tags?

Semantic tags clearly describe the purpose of their content.

Examples:

```
<header>
<nav>
<main>
<section>
<article>
<footer>
```

When someone reads the code, they immediately understand the role of each section.

Example

```
<article>
  <h2>Benefits of Exercise</h2>
  <p>Exercise improves physical and mental health.</p>
</article>
```

Even without reading the content, we know this is an article.

2. What are Non-Semantic Tags?

Non-semantic tags do not describe the meaning of their content.

Examples:


```
<div>
<span>
```

These tags are simply containers.

Example

```
<div>
  <h2>Benefits of Exercise</h2>
  <p>Exercise improves physical and mental health.</p>
</div>
```

Looking at the code, we don't know whether this is:

- an article
- a menu
- a sidebar
- a footer

It's just a container.

Why Semantic HTML Matters

1. Better Readability

Semantic HTML makes code easier to understand.

Non-semantic

```
<div class="top">
  <div class="menu">
    ...
  </div>
</div>
```

Semantic

```
<header>
  <nav>
    ...
  </nav>
</header>
```

The second version is much clearer.

2. Better SEO

Search engines like Google understand semantic tags.

Example:

```
<article>
```

Google knows this is actual content.

```
<nav>
```

Google knows these are navigation links.

This helps search engines understand your page structure.

3. Better Accessibility

Screen readers used by visually impaired users understand semantic tags.

For example:

```
<nav>
```

A screen reader can announce:

"Navigation section"

This makes websites easier to use.

Important Semantic Tags

1. `<main>`

Represents the main content of the page.

There should usually be only one `<main>` tag.

Example

```
<body>

<header>
  Website Header
</header>

<main>
  Main content goes here.
</main>

<footer>
  Footer
</footer>

</body>
```

Visual Structure

HEADER

MAIN CONTENT

FOOTER

2. `<nav>`

Represents navigation links.

Example

```
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
  <a href="/contact">Contact</a>
</nav>
```

Visual Structure

Home | About | Contact

Screen readers know this section contains navigation.

3. `<section>`

Represents a themed section of content.

Think of it as a chapter in a book.

Example

```
<section>
  <h2>About Us</h2>
  <p>We build software solutions.</p>
</section>
```

Another section:

```
<section>
  <h2>Services</h2>
  <p>Web Development</p>
</section>
```

Structure

```
ABOUT US
-----
content
```

```
SERVICES
-----
content
```

4. `<article>`

Represents independent content that can stand on its own.

Examples:

- Blog post
- News article
- Forum post
- Product review

Example

```
<article>
  <h2>How to Learn JavaScript</h2>
  <p>Practice every day...</p>
</article>
```

This article could be copied elsewhere and still make sense.

Article inside Section

```
<section>
  <h2>Latest Blogs</h2>

  <article>
    <h3>JavaScript Basics</h3>
  </article>

  <article>
    <h3>React Introduction</h3>
  </article>
</section>
```

Structure:

LATEST BLOGS

- JavaScript Basics
- React Introduction

<section> VS <article>

This is a common interview question.

Section

Used to group related content.

```
<section>
  <h2>Our Services</h2>
  ...
</section>
```

Article

Used for independent content.

```
<article>
  <h2>Blog Post</h2>
  ...
</article>
```

Easy Rule

Ask:

"Can this content exist independently?"

If **YES** → use `<article>`

If **NO** → use `<section>`

Non-Semantic Tags

1. `<div>`

A generic block-level container.

Used mainly for:

- Layout
- Styling
- Grouping elements

Example:

```
<div class="card">
  <h2>Product Name</h2>
  <p>Price: ₹999</p>
</div>
```

Without CSS, a `<div>` has no meaning.

2. `<span>`

A generic inline container.

Used for styling small pieces of text.

Example:

```
<p>
  Price:
  <span class="highlight">₹999</span>
</p>
```

The `<span>` only wraps the price.

Real-World Comparison

Non-Semantic Version

```
<div class="header">
  <div class="menu">
    Home
    About
  </div>
</div>

<div class="content">
  <div class="blog">
    Blog Post
  </div>
</div>
```

Everything is a `<div>`.

Semantic Version

```
<header>
  <nav>
    Home
    About
  </nav>
</header>

<main>
  <article>
    Blog Post
  </article>
</main>
```

Now anyone reading the code understands the page structure immediately.

Complete Example

```
<body>

<header>
  <h1>My Blog</h1>
</header>

<nav>
  <a href="#">Home</a>
  <a href="#">Blogs</a>
  <a href="#">Contact</a>
</nav>

<main>

  <section>
    <h2>Latest Articles</h2>

    <article>
      <h3>Learning HTML</h3>
      <p>HTML is the foundation of web development.</p>
    </article>

    <article>
      <h3>Learning CSS</h3>
      <p>CSS styles web pages.</p>
    </article>

  </section>

</main>

<footer>
  Copyright 2026
</footer>

</body>
```

Structure

```
HEADER
  My Blog

NAVIGATION
  Home
  Blogs
  Contact
```



```
MAIN
  LATEST ARTICLES
    Article 1
    Article 2
```

```
FOOTER
  Copyright
```

Interview Summary

Tag	Type	Purpose
<main>	Semantic	Main content of page
<nav>	Semantic	Navigation links
<section>	Semantic	Groups related content
<article>	Semantic	Independent content
<div>	Non-semantic	Generic block container
	Non-semantic	Generic inline container

One-Line Answer

Semantic tags describe the meaning and structure of content (<main>, <nav>, <section>, <article>), while non-semantic tags (<div>,) are generic containers used mainly for layout and styling without conveying any meaning about the content.

HTML Forms, Advanced Input Types, and Native Validation

Forms are one of the most important parts of web development because they allow users to enter and send data to a website.

Examples:

- Login page
- Registration page
- Contact form
- Feedback form
- Online shopping checkout form

1. What is an HTML Form?

An HTML form is used to collect user input.

Basic Syntax

```
<form>
  <label>Name:</label>
  <input type="text">

  <button type="submit">Submit</button>
</form>
```

Output

Name: [_____]

[Submit]

2. Main Form Elements

a) `<form>`

Acts as a container for all form fields.

```
<form>
```

```
...  
</form>
```

Important Attributes

Attribute	Purpose
action	Where data is sent
method	How data is sent
autocomplete	Enable/disable suggestions

Example:

```
<form action="/submit" method="POST">  
</form>
```

b) <label>

Provides a name for an input field.

```
<label for="username">Username</label>  
<input id="username" type="text">
```

Why use labels?

- Better accessibility
 - Clicking label focuses input
-

c) <input>

Used to take user input.

```
<input type="text">
```

Different type values create different input fields.

d) <textarea>

Used for multi-line text.

```
<textarea rows="4" cols="30"></textarea>
```

Example:

```
<textarea placeholder="Enter your message"></textarea>
```

Output:

e) `<select>` and `<option>`

Creates a dropdown list.

```
<select>
  <option>Mumbai</option>
  <option>Pune</option>
  <option>Delhi</option>
</select>
```

Output:

Mumbai ▼

f) `<button>`

Creates a clickable button.

```
<button>Click Me</button>
```

3. Common Input Types

Text Input

```
<input type="text">
```

Used for names, cities, etc.

Password Input

```
<input type="password">
```

Output:

Characters are hidden.

Email Input

```
<input type="email">
```

Browser checks email format.

Example:

user@gmail.com

Valid

usergmail.com

Invalid

Number Input

```
<input type="number">
```

Example:

```
<input type="number" min="1" max="100">
```

Only numbers allowed.

Checkbox

Used when multiple selections are allowed.

```
<input type="checkbox"> Cricket  
<input type="checkbox"> Football
```

Output:

☐ Cricket
☐ Football

Radio Button

Used when only one option should be selected.

```
<input type="radio" name="gender"> Male  
<input type="radio" name="gender"> Female
```

Output:

☐ Male
☐ Female

Only one can be selected because both share the same `name`.

File Upload

```
<input type="file">
```

Output:

Choose File

4. Advanced Input Types

HTML5 introduced many advanced input types.

These reduce JavaScript work and improve user experience.

Date Picker

```
<input type="date">
```

Output:

☐ Select Date

Used for:

- Birth dates
- Appointments
- Bookings

Time Picker

```
<input type="time">
```

Example:

09:30 AM

Date and Time

```
<input type="datetime-local">
```

Example:

31/05/2026 09:30

Month Picker

```
<input type="month">
```

Example:

May 2026

Week Picker

```
<input type="week">
```

Example:

Week 22, 2026

Color Picker

```
<input type="color">
```

Output:

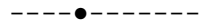


User selects a color.

Range Slider

```
<input type="range" min="0" max="100">
```

Output:



Used for:

- Volume controls
 - Brightness settings
 - Ratings
-

URL Input

```
<input type="url">
```

Valid:

```
https://google.com
```

Invalid:

```
google
```

Telephone Input

```
<input type="tel">
```


Used for phone numbers.

```
<input type="tel">
```

Search Input

```
<input type="search">
```

Used for search bars.

Example:

```
<input type="search" placeholder="Search">
```

5. Useful Input Attributes

Placeholder

Shows hint text.

```
<input type="text" placeholder="Enter your name">
```

Output:

Enter your name

Value

Sets default value.

```
<input type="text" value="Gaurav">
```

Readonly

User can view but not edit.

```
<input type="text" value="Admin" readonly>
```

Disabled

User cannot interact.

```
<input type="text" disabled>
```

Autofocus

Automatically focuses field when page loads.

```
<input type="text" autofocus>
```

Maxlength

Limits character count.

```
<input type="text" maxlength="10">
```

6. Native Form Validation

Native Validation = Browser validates input automatically without JavaScript.

Required Field

```
<input type="text" required>
```

If empty:

Please fill out this field.

Email Validation

```
<input type="email" required>
```

Wrong:

abcmail.com

Browser error:

Please include an '@' in the email address.

Number Validation

```
<input type="number" min="1" max="10">
```

Valid:

5

Invalid:

20

Browser shows validation error.

Pattern Validation

Allows custom format checking.

Example: 10-digit phone number.

```
<input  
type="text"  
pattern="[0-9]{10}"  
required>
```

Explanation:

[0-9]

Any digit

{10}

Exactly 10 digits

Valid:

9876543210

Invalid:

Min Length

```
<input
type="text"
minlength="5">
```

Must contain at least 5 characters.

Max Length

```
<input
type="text"
maxlength="15">
```

Cannot exceed 15 characters.

7. Complete Registration Form Example

```
<form>

  <label>Name</label>
  <input
    type="text"
    placeholder="Enter name"
    required>

  <br><br>

  <label>Email</label>
  <input
    type="email"
    required>

  <br><br>

  <label>Password</label>
  <input
    type="password"
    minlength="8"
    required>

  <br><br>
```

```
<label>Age</label>
<input
  type="number"
  min="18"
  max="60">

<br><br>

<label>Date of Birth</label>
<input type="date">

<br><br>

<label>Phone</label>
<input
  type="tel"
  pattern="[0-9]{10}"
  required>

<br><br>

<button type="submit">
  Register
</button>

</form>
```

What the browser validates automatically?

- ☑ Name cannot be empty
- ☑ Email must be valid
- ☑ Password must have at least 8 characters
- ☑ Phone must contain exactly 10 digits
- ☑ Age must be between 18 and 60

No JavaScript required.

Interview Questions

1. What is a form in HTML?

A form is used to collect and submit user data.

2. Difference between GET and POST?

GET	POST
Data visible in URL	Data hidden from URL
Less secure	More secure
Used for fetching	Used for sending

3. Why use `<label>`?

- Improves accessibility
 - Clicking label focuses input
-

4. Difference between checkbox and radio button?

Checkbox	Radio
Multiple selections	Single selection
Independent	Grouped by name

5. What is native validation?

Browser-provided validation without JavaScript.

Examples:

- `required`
 - `email`
 - `pattern`
 - `min`
 - `max`
 - `minlength`
 - `maxlength`
-

Quick Revision

Form Elements

```
<form>
<label>
<input>
<textarea>
```

```
<select>
<option>
<button>
```

Advanced Input Types

```
date
time
datetime-local
month
week
color
range
url
tel
search
```

Validation Attributes

```
required
min
max
minlength
maxlength
pattern
```

Most Important Interview Example

```
<input
type="email"
required>
```

This single line gives:

- ☑ Email validation
- ☑ Required validation
- ☑ No JavaScript needed

This is one of the best examples of HTML5 native form validation.

Web Accessibility (a11y): ARIA Attributes, Alt Text, and Keyboard Navigation

Accessibility (a11y) means designing websites so that **everyone can use them**, including people with disabilities.

The term **a11y** comes from:

- **A** = first letter
- **11** = number of letters between
- **Y** = last letter

Accessibility helps:

- Blind users (screen readers)
- Users with low vision
- Users who cannot use a mouse
- Users with hearing impairments
- Users with temporary injuries

1. What is a Screen Reader?

A **screen reader** is software that reads webpage content aloud.

Popular screen readers:

- NVDA
- JAWS
- VoiceOver

Example:

```
<h1>Welcome to My Website</h1>
```

A screen reader reads:

"Heading level 1, Welcome to My Website"

This helps visually impaired users understand the page.

2. Alt Text (Alternative Text)

What is Alt Text?

Alt text describes an image for people who cannot see it.

Syntax

```

```

Screen reader reads:

"A white cat sleeping on a sofa"

Why Alt Text is Important

Without alt text:

```

```

Screen reader may say:

"Image"

or

"cat dot jpg"

which is not useful.

Good Example

```

```

Describes what is actually visible.

Bad Example

```

```

Too vague.

Decorative Images

If an image is only for decoration:

```

```

Empty alt tells screen readers:

Ignore this image.

Example

Shopping Website

```

```

Now blind users know what product is displayed.

3. ARIA Attributes

What is ARIA?

ARIA = Accessible Rich Internet Applications

ARIA provides extra information to screen readers when HTML alone is not enough.

Why ARIA Exists

Sometimes we create custom UI elements:

```
<div>Submit</div>
```

Screen readers don't know this is a button.

ARIA can help.

aria-label

Provides a label for an element.

Example

```
<button aria-label="Close menu">  
  X  
</button>
```

Screen reader says:

"Close menu button"

instead of:

"X button"

Search Icon Example

```
<button aria-label="Search">  
    
</button>
```

Screen reader says:

"Search button"

aria-hidden

Hides content from screen readers.

Example

```
<span aria-hidden="true">★</span>
```

Screen reader ignores the star.

aria-required

Indicates a required field.

```
<input  
  type="text"  
  aria-required="true">
```

Screen reader says:

"Required"

aria-expanded

Used for dropdowns and menus.

Example

```
<button aria-expanded="false">  
  Menu  
</button>
```

Closed menu.

When opened:

```
<button aria-expanded="true">  
  Menu  
</button>
```

Screen reader knows the menu is expanded.

aria-describedby

Provides additional instructions.

```
<input
  type="password"
  aria-describedby="pwd-info">

<p id="pwd-info">
  Password must contain 8 characters.
</p>
```

Screen reader reads both the input and description.

Common ARIA Attributes

Attribute	Purpose
aria-label	Gives a label
aria-hidden	Hides from screen readers
aria-required	Marks required field
aria-expanded	Indicates open/closed state
aria-describedby	Provides extra description
aria-live	Announces updates automatically

aria-live Example

Useful for notifications.

```
<div aria-live="polite">
  Form submitted successfully.
</div>
```

Screen reader automatically announces the new message.

Important Rule

Use Semantic HTML First

Good:

```
<button>Submit</button>
```

Bad:

```
<div role="button">Submit</div>
```

Native HTML elements are already accessible.

Rule:

Use ARIA only when HTML cannot solve the problem.

4. Keyboard Navigation

Many users cannot use a mouse.

They navigate using:

- Tab
- Shift + Tab
- Enter
- Space
- Arrow keys

Your website must work completely from the keyboard.

Tab Navigation

Pressing **Tab** moves focus.

Example:

```
<input type="text">  
<button>Login</button>  
<a href="#">Forgot Password</a>
```

Tab order:

1. Input

2. Button
3. Link

Focus

The currently selected element is said to have focus.

Example:

```
<button>Submit</button>
```

When tabbed to:

```
[Submit]
```

A border appears around it.

This border is called a **focus indicator**.

Never Remove Focus Without Replacement

Bad:

```
button {  
  outline: none;  
}
```

Users won't know where they are.

Better:

```
button:focus {  
  outline: 2px solid blue;  
}
```

Making Custom Elements Keyboard Accessible

Bad:

```
<div onclick="submitForm()">
  Submit
</div>
```

Cannot be reached by keyboard.

Better:

```
<button onclick="submitForm()">
  Submit
</button>
```

Keyboard works automatically.

tabindex

Controls keyboard focus.

tabindex="0"

Adds element to tab order.

```
<div tabindex="0">
  Click Me
</div>
```

Now users can tab to it.

tabindex="-1"

Removes from normal tab order.

```
<div tabindex="-1">
  Hidden Focus Area
</div>
```

Can still receive focus through JavaScript.

Avoid Large Positive Values

Bad:

```
<div tabindex="100">
```

Creates confusing navigation.

Keyboard Accessible Modal Example

```
<button>Open Modal</button>
```

```
<div role="dialog">
  <h2>Login</h2>

  <input type="text">
  <button>Submit</button>
</div>
```

Users should be able to:

- Tab through controls
 - Press Enter
 - Press Escape to close
-

Accessibility Example: Login Form

```
<form>

  <label for="email">
    Email
  </label>

  <input
    id="email"
    type="email"
    required>

  <label for="password">
    Password
  </label>

  <input
    id="password"
    type="password">
```

```
        required>

        <button>
            Login
        </button>

    </form>
```

Why this is accessible:

- ☑ Uses labels
 - ☑ Uses semantic HTML
 - ☑ Works with keyboard
 - ☑ Screen readers understand it
 - ☑ Required fields are announced
-

Accessibility Checklist

Before publishing a webpage, check:

Images

- ☑ Every meaningful image has alt text
- ☑ Decorative images use `alt=""`

Forms

- ☑ Inputs have labels
- ☑ Required fields are marked

Keyboard

- ☑ Everything works using Tab and Enter
- ☑ Focus is visible

ARIA

- ☑ Use only when necessary
- ☑ Use correct ARIA attributes

HTML

- ☑ Prefer semantic elements:

- `<header>`
 - `<main>`
 - `<nav>`
 - `<section>`
 - `<article>`
 - `<button>`
 - `<form>`
-

Interview Questions

1. What is web accessibility?

Web accessibility is the practice of making websites usable for everyone, including people with disabilities.

2. What is alt text?

Alternative text that describes an image for screen readers and when images fail to load.

3. What is ARIA?

ARIA (Accessible Rich Internet Applications) is a set of attributes that provide additional accessibility information to assistive technologies.

4. What is `aria-label`?

It provides an accessible name for an element.

Example:

```
<button aria-label="Close">  
  X  
</button>
```

5. What is keyboard navigation?

Using the keyboard (Tab, Enter, Space, Arrow keys) to interact with a website without a mouse.

6. Why should we prefer semantic HTML over ARIA?

Semantic HTML elements are accessible by default, simpler, and more reliable. ARIA should only be used when native HTML cannot provide the needed accessibility.

Quick Summary

- **Alt Text** → Describes images for screen readers.
- **ARIA Attributes** → Provide extra accessibility information when HTML alone isn't enough.
- **Keyboard Navigation** → Ensures users can use the entire website without a mouse.
- **Semantic HTML First, ARIA Second** → Native HTML elements are usually the best accessibility solution.
- **Visible Focus + Proper Labels + Alt Text** are the foundation of accessible web pages.

SEO Basics: Meta Tags, Open Graph Protocol, and Semantic Structure Hierarchy

SEO (**Search Engine Optimization**) is the process of improving a website so that search engines like Google can understand it better and show it higher in search results.

Think of SEO like this:

- Your website = a book
 - Search engine = a librarian
 - SEO = labels and organization that help the librarian understand the book
-

1. Meta Tags

What are Meta Tags?

Meta tags are pieces of information placed inside the `<head>` section of an HTML page.

They are usually **not visible to users**, but they help:

- Search engines understand the page
 - Browsers display information correctly
 - Social media platforms show previews
-

Basic Structure

```
<head>
  <meta charset="UTF-8">
  <meta name="description" content="Learn HTML, CSS, and JavaScript">
  <meta name="keywords" content="HTML, CSS, JavaScript">
  <meta name="author" content="John">
</head>
```

Important Meta Tags

1. Title Tag

The most important SEO tag.

```
<title>Learn HTML Basics</title>
```

Shows in:

- Browser tab
- Google search results

Example:

```
Learn HTML Basics  
www.example.com  
Learn HTML from scratch...
```

2. Meta Description

Provides a short summary of the page.

```
<meta  
  name="description"  
  content="Learn HTML, CSS, and JavaScript with simple examples."  
>
```

Google may show this description under the page title.

Example:

```
Learn HTML, CSS, and JavaScript with simple examples.
```

3. Meta Keywords

```
<meta  
  name="keywords"  
  content="HTML, CSS, JavaScript"  
>
```

Historically used for SEO.

Today, most search engines ignore it, but you may still see it on websites.

4. Viewport Tag

Makes websites responsive on mobile devices.

```
<meta  
  name="viewport"
```

```
    content="width=device-width, initial-scale=1.0"
>
```

Without it:

✗ Website may look zoomed out on phones.

With it:

✓ Website adjusts properly.

5. Charset

Defines character encoding.

```
<meta charset="UTF-8">
```

Supports:

- English
 - Hindi
 - Marathi
 - Emoji ☐
-

Example of a Good Head Section

```
<head>
  <title>JavaScript Interview Questions</title>

  <meta charset="UTF-8">

  <meta
    name="description"
    content="Top JavaScript interview questions with answers."
  >

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
</head>
```

2. Open Graph (OG) Protocol

What is Open Graph?

Open Graph controls how your page appears when shared on social media platforms.

Created by [Facebook Open Graph Protocol](#).

Without Open Graph:

`example.com/article`

Only a plain link appears.

With Open Graph:

[Image]
Best JavaScript Tips
Learn useful JS tips...

A rich preview appears.

Common Open Graph Tags

OG Title

```
<meta
  property="og:title"
  content="Best JavaScript Tips"
>
```

OG Description

```
<meta
  property="og:description"
  content="Learn useful JavaScript tricks."
>
```

OG Image

```
<meta
  property="og:image"
  content="https://example.com/image.jpg"
>
```

This image appears in the preview.

OG URL

```
<meta
  property="og:url"
  content="https://example.com"
>
```

OG Type

```
<meta
  property="og:type"
  content="website"
>
```

Common values:

- website
 - article
 - video
 - product
-

Complete Example

```
<head>

<meta property="og:title"
content="JavaScript Interview Guide">

<meta property="og:description"
content="Most asked JavaScript questions.">

<meta property="og:image"
content="https://example.com/js.png">

<meta property="og:url"
content="https://example.com">

<meta property="og:type"
content="website">

</head>
```

When Someone Shares Your Link

Without OG Tags:

`https://example.com`

With OG Tags:

[Image]

JavaScript Interview Guide

Most asked JavaScript questions.

Much more attractive and clickable.

3. Semantic Structure Hierarchy

What is Semantic HTML?

Semantic HTML uses tags that describe the meaning of content.

Instead of:

`<div>`

Use:

`<header>`

`<nav>`

`<main>`

`<footer>`

These tags tell browsers and search engines what each section represents.

Why Important for SEO?

Search engines can understand:

- What is navigation
- What is the main content
- What is an article
- What is a footer

Better understanding = better indexing.

Common Semantic Tags

1. Header

Top section of page.

```
<header>
  <h1>My Blog</h1>
</header>
```

Contains:

- Logo
 - Website name
 - Navigation
-

2. Nav

Navigation menu.

```
<nav>
  <a href="/">Home</a>
  <a href="/about">About</a>
</nav>
```

Search engines recognize these as navigation links.

3. Main

Main content area.

```
<main>
  Main content here
</main>
```

Only one `<main>` should exist per page.

4. Section

Groups related content.

```
<section>
  <h2>Latest Posts</h2>
</section>
```

Think of it as a chapter in a book.

5. Article

Independent content.

```
<article>
  <h2>HTML Basics</h2>
  <p>Content...</p>
</article>
```

Examples:

- Blog post
 - News article
 - Forum post
-

6. Aside

Extra content.

```
<aside>
  Related Articles
</aside>
```

Examples:

- Ads
 - Sidebar
 - Related posts
-

7. Footer

Bottom section.

```
<footer>
  Copyright 2026
</footer>
```

Contains:

- Contact information
- Copyright
- Social links

Proper Semantic Hierarchy

Good Structure:

```
<body>

<header>
  <h1>My Blog</h1>
</header>

<nav>
  Menu Links
</nav>

<main>

  <section>

    <article>
      Blog Post 1
    </article>

    <article>
      Blog Post 2
    </article>

  </section>

</main>

<aside>
  Related Posts
</aside>

<footer>
```

Copyright
</footer>

</body>

Heading Hierarchy (Very Important for SEO)

Use headings in order.

✓ Good:

```
<h1>JavaScript Guide</h1>
```

```
<h2>Variables</h2>
```

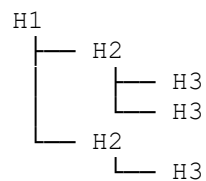
```
<h3>let</h3>
```

```
<h3>const</h3>
```

```
<h2>Functions</h2>
```

```
<h3>Arrow Functions</h3>
```

Structure:



✗ Bad:

```
<h1>JavaScript Guide</h1>
```

```
<h4>Variables</h4>
```

```
<h2>Functions</h2>
```

This confuses search engines.

Real-World SEO-Friendly Page

```
<!DOCTYPE html>
<html>

<head>

<title>JavaScript Interview Questions</title>

<meta charset="UTF-8">

<meta name="description"
content="Top JavaScript interview questions">

<meta name="viewport"
content="width=device-width, initial-scale=1.0">

<meta property="og:title"
content="JavaScript Interview Questions">

<meta property="og:description"
content="Most asked JS interview questions">

<meta property="og:type"
content="website">

</head>

<body>

<header>
  <h1>JavaScript Interview Questions</h1>
</header>

<nav>
  Navigation Menu
</nav>

<main>

  <section>

    <article>

      <h2>Variables</h2>

      <p>Learn about let, const and var.</p>

    </article>

  </section>

</main>

<footer>
  Copyright 2026
</footer>

</body>
```

</html>

Interview Quick Revision

Meta Tags

- Located inside <head>
- Help search engines understand pages
- Important tags:
 - <title>
 - description
 - viewport
 - charset

Open Graph Protocol

- Controls social media link previews
- Common tags:
 - og:title
 - og:description
 - og:image
 - og:url
 - og:type

Semantic Structure Hierarchy

- Uses meaningful tags:
 - <header>
 - <nav>
 - <main>
 - <section>
 - <article>
 - <aside>
 - <footer>
- Improves SEO and accessibility
- Use proper heading order:
 - h1 → h2 → h3 → h4

Easy Memory Trick

SEO = Content + Structure + Metadata

- **Content** → Articles, headings, text
- **Structure** → Semantic HTML (header, main, article)
- **Metadata** → Title, description, Open Graph tags

If you're preparing for frontend interviews, these three topics are among the most frequently asked HTML/SEO fundamentals.

This is a great exercise because it combines **HTML**, **semantic structure**, **accessibility (a11y)**, and **real-world interview preparation**.

Project Goal

Build a **multi-page documentation website** using only HTML (no CSS or JavaScript required).

Example: Documentation website for **HTML Basics**.

Pages:

1. Home (`index.html`)
2. HTML Basics (`basics.html`)
3. Forms (`forms.html`)
4. Accessibility (`accessibility.html`)
5. Contact (`contact.html`)

Step 1: Create Project Folder

Create a folder:

```
documentation-site/  
|  
├─ index.html  
├─ basics.html  
├─ forms.html  
├─ accessibility.html  
└─ contact.html
```

Step 2: Create the Home Page

Create `index.html`

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <title>HTML Documentation</title>
```

```
</head>
<body>

<header>
  <h1>HTML Documentation Site</h1>
</header>

<nav aria-label="Main Navigation">
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="basics.html">HTML Basics</a></li>
    <li><a href="forms.html">Forms</a></li>
    <li><a href="accessibility.html">Accessibility</a></li>
    <li><a href="contact.html">Contact</a></li>
  </ul>
</nav>

<main>
  <section>
    <h2>Welcome</h2>
    <p>Learn HTML from beginner to advanced.</p>
  </section>
</main>

<footer>
  <p>© 2026 HTML Docs</p>
</footer>

</body>
</html>
```

Step 3: Add Semantic Tags

Use semantic tags everywhere.

Instead of:

```
<div>
  Content
</div>
```

Use:

```
<header>
<nav>
<main>
<section>
<article>
<footer>
```

Example:

```
<main>
  <article>
    <h2>Introduction to HTML</h2>
    <p>HTML is the structure of a webpage.</p>
  </article>
</main>
```

This improves:

- Accessibility
- SEO
- Readability

Step 4: Create the HTML Basics Page

basics.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>HTML Basics</title>
</head>
<body>

<header>
  <h1>HTML Basics</h1>
</header>

<nav aria-label="Main Navigation">
```

```
<a href="index.html">Home</a>
</nav>

<main>

<section>
  <h2>What is HTML?</h2>
  <p>HTML stands for HyperText Markup Language.</p>
</section>

<section>
  <h2>Basic Structure</h2>

  <pre>
    &lt;html&gt;
    &lt;head&gt;&lt;/head&gt;
    &lt;body&gt;&lt;/body&gt;
    &lt;/html&gt;
  </pre>

</section>

</main>

<footer>
  <p>End of page</p>
</footer>

</body>
</html>
```

Step 5: Create Forms Page

forms.html

```
<main>

<h1>Forms</h1>

<form>
```

```
<label for="name">Name</label>
<input id="name" type="text">

<br><br>

<label for="email">Email</label>
<input id="email" type="email">

<br><br>

<button type="submit">
Submit
</button>

</form>

</main>
```

Accessibility points:

☑ Labels connected with inputs

```
<label for="email">Email</label>
<input id="email">
```

Step 6: Create Accessibility Page

accessibility.html

```
<main>

<h1>Accessibility</h1>

<section>
  <h2>Alt Text</h2>

  
</section>
```

```
<section>
  <h2>ARIA Example</h2>

  <button aria-label="Close Menu">
    X
  </button>

</section>

</main>
```

Step 7: Create Contact Page

contact.html

```
<main>

  <h1>Contact Us</h1>

  <form>

    <label for="message">
      Message
    </label>

    <textarea
      id="message">
    </textarea>

    <br><br>

    <button>
      Send
    </button>

  </form>

</main>
```

Step 8: Accessibility Checklist

Before submission check:

1. Every image has alt text

✓ Good

```

```

✗ Bad

```

```

2. Form fields have labels

✓ Good

```
<label for="name">Name</label>
<input id="name">
```

3. Proper heading order

✓ Good

```
<h1>Main Title</h1>
<h2>Section</h2>
<h3>Subsection</h3>
```

✗ Bad

```
<h1>Main</h1>
<h4>Section</h4>
```

4. Use semantic tags

```
<header>
<nav>
<main>
<section>
<footer>
```

5. Use ARIA only when necessary

Example:

```
<nav aria-label="Main Navigation">
```

Step 9: Validate with W3C Validator

Open:

[W3C Markup Validation Service](https://validator.w3.org/)

Method 1

1. Open page.
 2. Copy all HTML.
 3. Click "Validate by Direct Input".
 4. Paste code.
 5. Click Check.
-

Fix Any Errors

Example:

Error:

```

```

Fix:

```

```

Keep fixing until:

```
Document checking completed.  
No errors or warnings to show.
```

Interview Question 1: Why Use Semantic Tags?

Answer

Semantic tags describe the meaning of content instead of just its appearance.

Examples:

```
<header>
<nav>
<main>
<footer>
```

Benefits:

1. Better accessibility for screen readers.
2. Better SEO.
3. Easier code maintenance.
4. Clearer page structure.

Interview Example

Instead of:

```
<div class="header">
```

Use:

```
<header>
```

because it clearly tells browsers and assistive technologies that this is the page header.

Interview Question 2: What is the Purpose of aria-label?

Answer

`aria-label` provides an accessible name for an element when visible text is missing or insufficient.

Example:

```
<button aria-label="Close Menu">
  X
</button>
```

A screen reader will announce:

```
Close Menu button
```

instead of:

```
X button
```

Common Uses

Search icon:

```
<button aria-label="Search">  
    
</button>
```

Menu icon:

```
<button aria-label="Open Navigation Menu">  
    
</button>
```

Close icon:

```
<button aria-label="Close Dialog">  
    
</button>
```

What Interviewers Expect You to Say

Semantic Tags:

"Semantic tags give meaning to content, improve accessibility, SEO, and code readability."

aria-label:

"aria-label provides an accessible name for elements when visible text is not enough, helping screen-reader users understand the element's purpose."